

6.3 Reading Lines

It's often convenient for a program to process its input not a character at a time but rather a line at a time, that is, to read an entire line of input and then act on it all at once. The standard C library has a couple of functions for reading lines, but they have a few awkward features, so we're going to learn more about character input (and about writing functions in general) by writing our own function to read one line. Here it is:

```
#include <stdio.h>

/* Read one line from standard input, */
/* copying it to line array (but no more than max chars). */
/* Does not place terminating \n in line array. */
/* Returns line length, or 0 for empty line, or EOF for end-of-file. */

int getline(char line[], int max)
{
    int nch = 0;
    int c;
    max = max - 1;                /* leave room for '\0' */

    while((c = getchar()) != EOF)
    {
        if(c == '\n')
            break;

        if(nch < max)
        {
            line[nch] = c;
            nch = nch + 1;
        }
    }

    if(c == EOF && nch == 0)
        return EOF;

    line[nch] = '\0';
    return nch;
}
```

As the comment indicates, this function will read one line of input from the standard input, placing it into the `line` array. The size of the `line` array is given by the `max` argument; the function will never write more than `max` characters into `line`.

The main body of the function is a `getchar` loop, much as we used in the character-copying program. In the body of this loop, however, we're storing the characters in an array (rather than immediately printing them out). Also, we're only reading one line of characters, then stopping and returning.

There are several new things to notice here.

First of all, the `getline` function accepts an array as a parameter. As we've said, array parameters are an exception to the rule that functions receive copies of their arguments--in the case of arrays, the function *does* have access to the actual array passed by the caller, and *can* modify it. Since the function is accessing the caller's array, not creating a new one to hold a copy, the function does not have to declare the argument array's size; it's set by the caller. (Thus, the brackets in `char line[]` are empty.) However, so that we won't overflow the caller's array by reading too long a line into it, we allow the caller to pass along the size of the array, which we promise not to exceed.

Second, we see an example of the `break` statement. The top of the loop looks like our earlier character-copying loop--it stops when it reaches `EOF`--but we only want this loop to read one line, so we also stop (that is, break out of the loop) when we see the `\n` character signifying end-of-line. An equivalent loop, without the `break` statement, would be

```
while((c = getchar()) != EOF && c != '\n')
{
    if(nch < max)
    {
        line[nch] = c;
        nch = nch + 1;
    }
}
```

We haven't learned about the internal representation of strings yet, but it turns out that strings in C are simply arrays of characters, which is why we are reading the line into an array of characters. The end of a string is marked by the special character, `'\0'`. To make sure that there's always room for that character, on our way in we subtract 1 from `max`, the argument that tells us how many characters we may place in the `line` array. When we're done reading the line, we store the end-of-string character `'\0'` at the end of the string we've just built in the `line` array.

Finally, there's one subtlety in the code which isn't too important for our purposes now but which you may wonder about: it's arranged to handle the possibility that a few characters (i.e. the apparent beginning of a line) are read, followed immediately by an `EOF`, without the usual `\n` end-of-line character. (That's why we return `EOF` only if we received `EOF` *and* we hadn't read any characters first.)

In any case, the function returns the length (number of characters) of the line it read, not including the `\n`. (Therefore, it returns 0 for an empty line.) Like `getchar`, it returns `EOF` when there are no more lines to read. (It happens that `EOF` is a negative number, so it will never match the length of a line that `getline` has read.)

Here is an example of a test program which calls `getline`, reading the input a line at a time and then printing each line back out:

```
#include <stdio.h>

extern int getline(char [], int);

main()
{
    char line[256];

    while(getline(line, 256) != EOF)
        printf("you typed \"%s\"\n", line);

    return 0;
}
```

The notation `char []` in the function prototype for `getline` says that `getline` accepts as its first argument an array of `char`. When the program calls `getline`, it is careful to pass along the actual size of the array. (You might notice a potential problem: since the number 256 appears in two places, if we ever decide that 256 is too small, and that we want to be able to read longer lines, we could easily change one of the instances of 256, and forget to change the other one. Later we'll learn ways of solving--that is, avoiding--this sort of problem.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)